

Report 6: Convolutional Neural Network

Hoai Nam Le

May 23, 2026

1 Network Architecture

A convolutional neural network (CNN) was implemented from scratch using NumPy and OpenCV.

The network architecture contains:

- Convolution layer
- ReLU activation
- Max pooling layer
- Fully connected layer
- Softmax output layer

Architecture:

Input 28x28 -> Convolution 3x3 -> ReLU -> MaxPooling 2x2 -> Flatten
-> Fully Connected -> Softmax

The convolution layer extracts image features using a learnable kernel.

$$output(i, j) = \sum region \times kernel$$

The ReLU activation introduces non-linearity:

$$ReLU(x) = \max(0, x)$$

Max pooling reduces the spatial size while keeping important features.

The fully connected layer performs classification.

Softmax converts outputs into probabilities:

$$Softmax(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

2 Training Pipeline

The CNN training pipeline is:

Image -> Convolution -> ReLU -> Pooling -> Flatten -> Fully Connected Softmax
-> Cross Entropy Loss -> Backpropagation -> Gradient Descent

The model was trained using gradient descent.
Weights were updated using:

$$W = W - r \nabla W$$

where:

- W is the weight matrix
- r is the learning rate
- ∇W is the gradient

Cross entropy loss was used:

$$Loss = -\log(p_y)$$

3 Dataset

The MNIST handwritten digit dataset was used.
The dataset contains 10 classes:

$$0, 1, 2, \dots, 9$$

Each image has size:

$$28 \times 28$$

4 Evaluation Metrics

The network was evaluated using classification metrics.
Accuracy measures the percentage of correct predictions.

$$Accuracy = \frac{CorrectPredictions}{TotalPredictions}$$

Experimental result:

Accuracy = 0.84

5 Discussion

The CNN architecture successfully implemented:

- Convolution
- Pooling
- Softmax classification
- Backpropagation
- Gradient descent

The loss decreased during training, showing that the model learned from the dataset.

The accuracy is still limited because:

- Only one convolution kernel was used
- Only the fully connected layer was trained
- The network architecture is simplified

6 Training Result Figure

```
namele@fedora:~/dl2026/Lab6
~/dl2026/Lab6
namele@fedora:~/dl2026/Lab6 x namele@fedora:~/dl2026
Epoch 60, Loss = 0.9648
Epoch 61, Loss = 0.9574
Epoch 62, Loss = 0.9501
Epoch 63, Loss = 0.9429
Epoch 64, Loss = 0.9360
Epoch 65, Loss = 0.9292
Epoch 66, Loss = 0.9225
Epoch 67, Loss = 0.9160
Epoch 68, Loss = 0.9097
Epoch 69, Loss = 0.9035
Epoch 70, Loss = 0.8974
Epoch 71, Loss = 0.8915
Epoch 72, Loss = 0.8856
Epoch 73, Loss = 0.8800
Epoch 74, Loss = 0.8744
Epoch 75, Loss = 0.8689
Epoch 76, Loss = 0.8636
Epoch 77, Loss = 0.8583
Epoch 78, Loss = 0.8532
Epoch 79, Loss = 0.8482
Epoch 80, Loss = 0.8433
Epoch 81, Loss = 0.8384
Epoch 82, Loss = 0.8337
Epoch 83, Loss = 0.8290
Epoch 84, Loss = 0.8245
Epoch 85, Loss = 0.8200
Epoch 86, Loss = 0.8156
Epoch 87, Loss = 0.8113
Epoch 88, Loss = 0.8070
Epoch 89, Loss = 0.8029
Epoch 90, Loss = 0.7988
Epoch 91, Loss = 0.7948
Epoch 92, Loss = 0.7908
Epoch 93, Loss = 0.7870
Epoch 94, Loss = 0.7831
Epoch 95, Loss = 0.7794
Epoch 96, Loss = 0.7757
Epoch 97, Loss = 0.7721
Epoch 98, Loss = 0.7685
Epoch 99, Loss = 0.7650
Epoch 100, Loss = 0.7616
Accuracy = 0.84
(dl2026) namele@fedora:~/dl2026/Lab6$
```

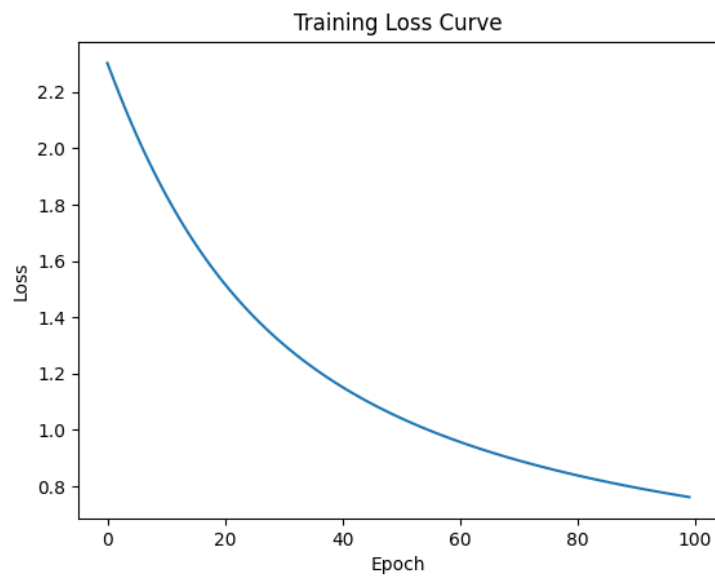


Figure 1: CNN training result and accuracy